Announcement

- 期中考
  - 4/22 (三)14:10 ~ 16:00
  - 地點: 337, 338 (請參考ilearning 公告)
  - 範圍C1~C5
  - No open book
  - 考試開始30分鐘後不得入場
- Office hour
  - 4/20 18:30-21:00 at this class
  - 若有課程相關問題可以前來
- 期末專題
- 學習活動(課堂練習)

Deep Learning Models

Why Move from Traditional NLP to Deep Learning?

- Traditional NLP relies on hand-crafted features (e.g., Bag-of-Words, TF-IDF).
  - Formula is designed by human
- Often fails to capture context, word order, or semantics.
- Struggles with sarcasm, polysemy (e.g., "bank"), and long-distance dependencies.
- Deep learning allows automatic feature learning from data.
- Enables more powerful, context-aware models.

Why Deep Learning Works Well in NLP

- Learns word meaning through dense vectors (embeddings).
- Captures syntax, semantics, and contextual relationships.
- Scales to large corpora and diverse domains.
- Enables end-to-end learning from raw input to prediction.
- Flexible and adaptable across NLP tasks.

What to Watch Out for When Using Deep Learning in NLP

- **Data Hungry**: Requires large, labeled datasets for training.
- **Computationally Expensive**: Needs powerful hardware (GPU/TPU).
- **Interpretability**: Deep models are often black boxes.
- **Overfitting**: Especially with small or biased data.
- **Ethical Issues**: May reproduce or amplify biases in data.

Preprocessing Considerations in Deep Learning NLP

Stemming and Lemmatization

- Sometimes skipped in DL pipelines due to learned representations
  - 深度學習模型能學到 running、runs、ran 在空間中複雜關係，強行把它們都砍成 run，反而可能破壞了原本細微的語態資訊

- May still help in low-resource settings or when training from scratch
  - 如果資料量很小，深度學習模型可能無法自行學到複雜的時態變化，這時或許就可以手動做一些處理

Stopword Removal

- Sometimes skipped in DL pipelines due to learned representations
- May still help in low-resource settings or when training from scratch
- Optional; models like BERT often benefit from keeping them
  - 停用詞通常會帶有重要的關係資訊
  - Ex: “Flight from Paris to London” vs “Flight to Paris from London”

Padding and Truncation

- In Tensorflow, input sequences must all have equal length
- Use special tokens (e.g., [PAD] or 0)
- padded_seq = pad_sequences(unpadded_seq)
- RNNs have trouble “remembering” the past, so it’s usually more useful to have padding at the front

```
unpadded_sequences = [
    [1, 2, 3, 4, 5],
    [1, 6, 7, 4, 8],
    [1, 9, 10]
]
```

GPU 無法處理這個3x?的矩陣，除非將batch size改為1

```
padded_sequences = [
    [1, 2, 3, 4, 5],
    [1, 6, 7, 4, 8],
    [0, 0, 1, 9, 10]
]
```

Shapes

- An array containing padded sequences of ints representing sentences will have the shape NxT
- N = # of documents or batch size, T = max document length

```
padded_sequences = [
    [1, 2, 3, 4, 5],
    [1, 6, 7, 4, 8],
    [0, 0, 1, 9, 10]
]
```

Embedding

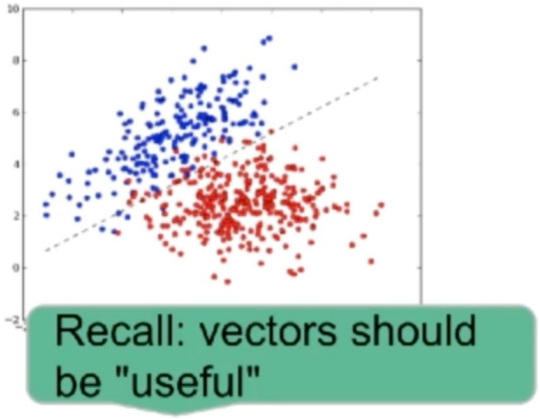
- Consider a generic task, like trying to predict whether or not a user will purchase something on your website
- One data point you have is how the user got to your site:
  - Search (e.g. google)
  - Advertisement
  - Friend
- Can you just assign numbers? 0 = Search, 1 = Ad, 2 = Friend? No…
- One-hot encoding: [1,0,0] = Search, [0,1,0] = Ad, [0,0,1] = Friend
- If w_1 is large/positive, search users are more likely to purchase
- If w_2 is large/negative, ad users are less likely to purchase

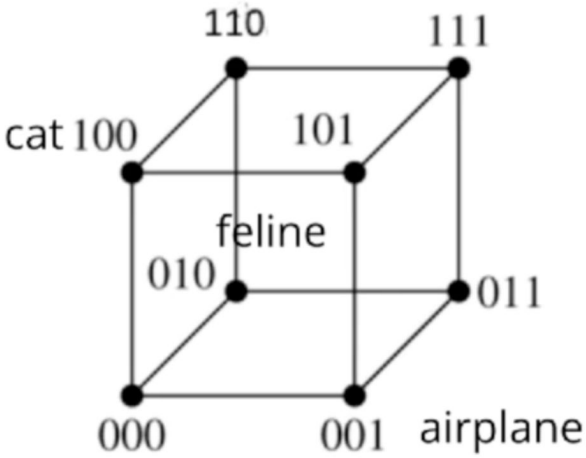
Same Strategy for NLP

- Treat each word as a category, and create a big feature vector with one position for every possible word, e.g.
  [1,0,0, …, 0] = “a”
  [0,1,0, …, 0] = “aa”
  …
  [0,0,0, …, 1] = “zygote”
- We can then do our usual $W^T x + b$ computation where x = one-hot vector

Is One-Hot Encoding Enough?

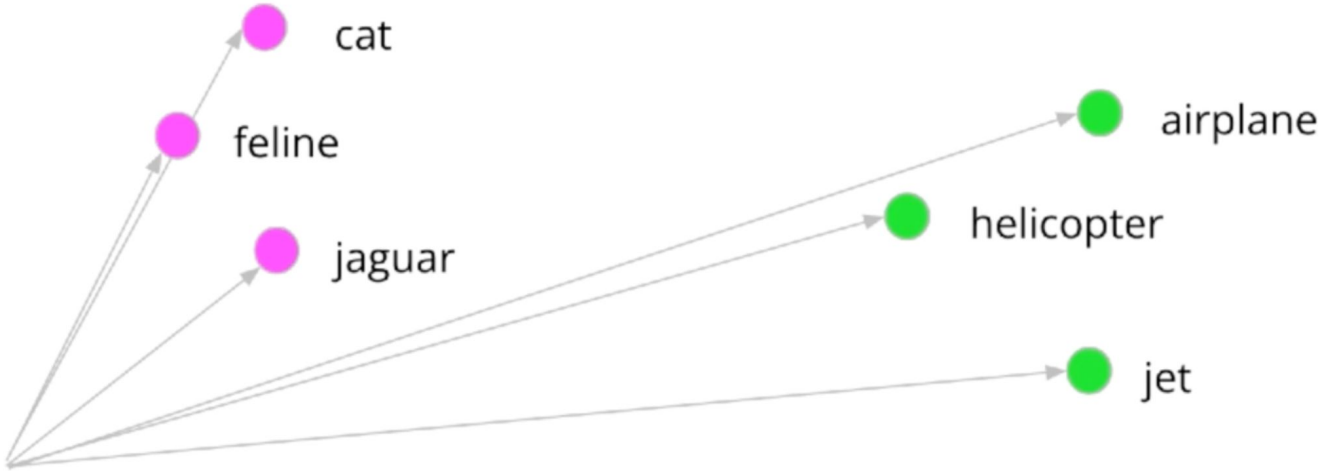
- One-hot encoded vectors are not “useful” geometrically





Big Picture

- What we've effectively done is created a table / database of word vectors for each word - word index is a query into the table
- We hope that these vectors will have some useful structure (unlike one-hot vectors)



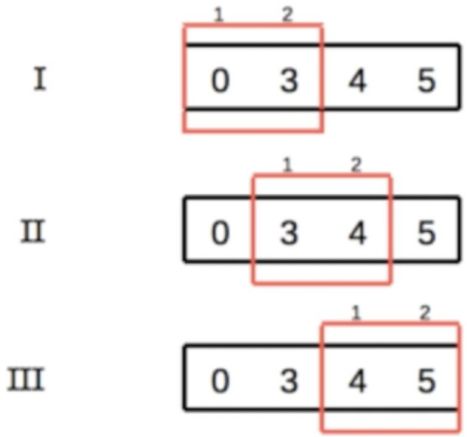
Other Embedding Approaches

- Word2Vec
  - Learn from predicting context
- GloVe
  - Learn from word co-occurrence counts
- Train your own embedding layer
  - Embedding(V, D) in Keras / PyTorch
  - Learns embeddings directly from your task data
- FastText
  - Subword-based embedding
  - Breaks words into character n-grams
- Contextual Embeddings (BERT, GPT)
  - Embeddings change based on sentence context

CNN x NLP

1-D Convolution

- Slide the filter along every position, multiply and add



1-D Convolution Example

- Input sequence

1	2	3	2	1

- Filter

1	-1

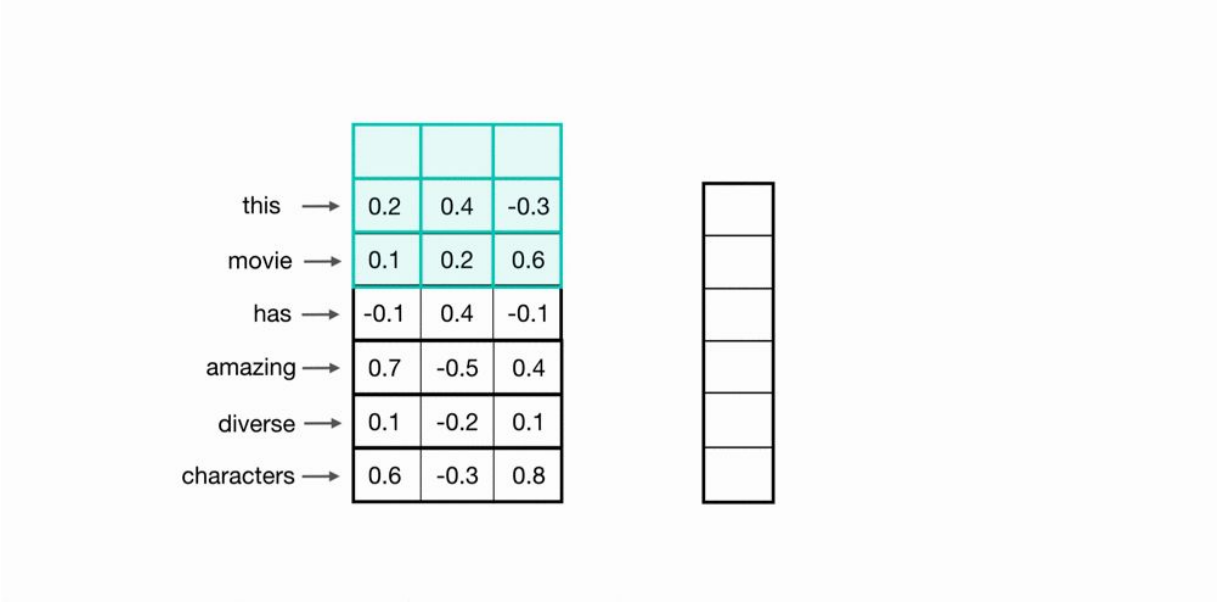
- Result

-1	-1	1	1

https://cezannec.github.io/CNN_Text_Classification/

Convolution on Text

- Using 1-D convolution



https://cezannec.github.io/CNN_Text_Classification/

1-D Convolutional Kernels

width =
length of embedding

height =
numbers of words
to look at in sequence

0.5	0.4	0.7
0.2	-0.1	0.3

Note: you don't have to worry about the width (e.g. embedding size) when creating a 1-D Conv layer, just as you don't need to deal with the depth (e.g. input channels in images) when using a 2-D Conv layer

https://cezannec.github.io/CNN_Text_Classification/

Recognizing General Patterns

the →	0.2	0.4	-0.1
good →	0.7	-0.5	0.3
movie →	0.1	0.2	0.6

0.5	0.4	0.7
0.2	-0.1	0.3

$$0.5 * 0.7 + 0.4 * -0.5 + 0.7 * 0.3 + 0.2 * 0.1 + -0.1 * 0.2 + 0.3 * 0.6 = 0.54$$

a →	0.1	0.3	-0.2
fantastic →	0.8	-0.6	0.3
song →	0.2	0.3	0.5

$$0.5 * 0.8 + 0.4 * -0.6 + 0.7 * 0.3 + 0.2 * 0.2 + -0.1 * 0.3 + 0.3 * 0.5 = 0.53$$

Found pattern: a positive thing

CNN for Text in Code

```
i = Input(shape=(T,))
x = Embedding(V + 1, D)(i)
x = Conv1D(32, 3, activation='relu')(x)
x = MaxPooling1D(3)(x)
x = Conv1D(64, 3, activation='relu')(x)
x = MaxPooling1D(3)(x)
x = Conv1D(128, 3, activation='relu')(x)
x = GlobalMaxPooling1D()(x)
# OR: x = Flatten()(x)
x = Dense(1)(x)
# OR: x = Dense(K)(x)
```

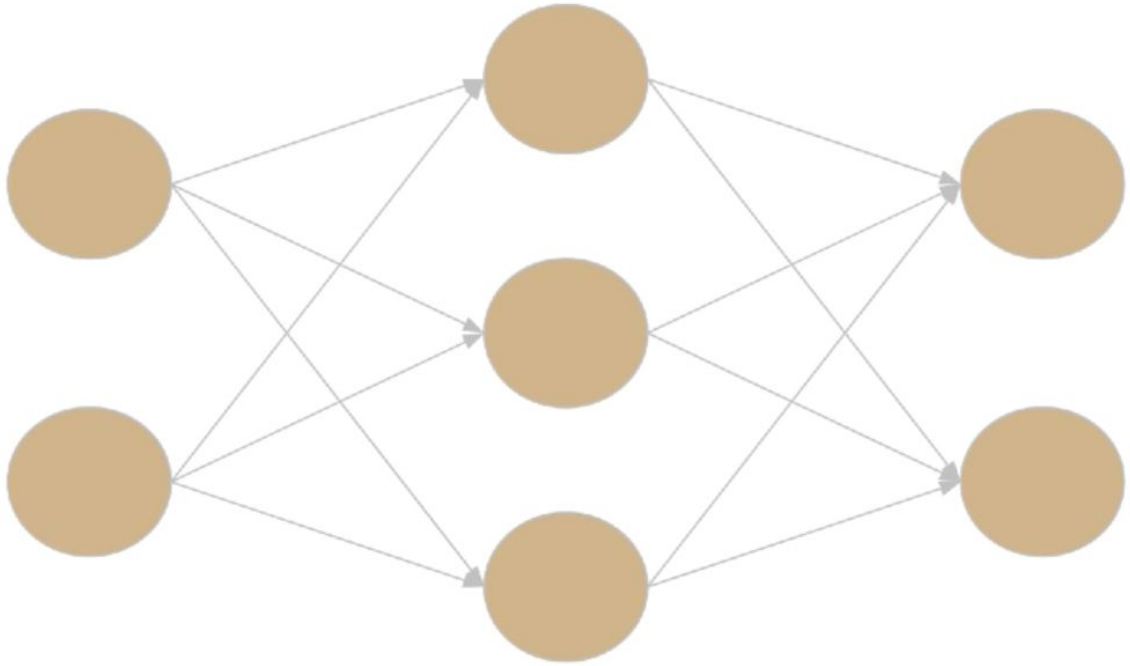
Layer	Output Shape	Description
Input	(batch_size, 100)	Token ID 序列
Embedding	(batch_size, 100, 50)	詞向量
Conv1D (32 filters)	(batch_size, 98, 32)	區域特徵
MaxPool1D(3)	(batch_size, 32, 32)	降維
Conv1D (64 filters)	(batch_size, 30, 64)	更深層特徵
MaxPool1D(3)	(batch_size, 10, 64)	再降維
Conv1D (128 filters)	(batch_size, 8, 128)	更深層次抽象
GlobalMaxPool1D	(batch_size, 128)	變成固定長度的句子表示
Dense(1)	(batch_size, 1)	預測結果（如情感分類）

RNN x NLP

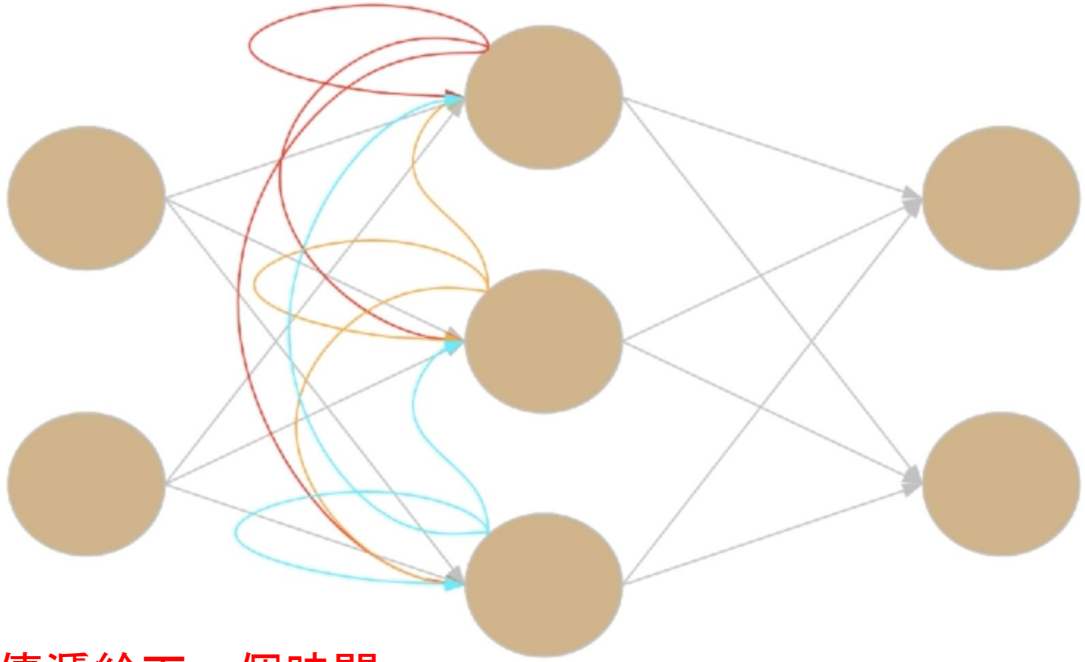
Why CNN Is Not Enough

- CNNs are great for capturing local patterns in text, such as short phrases like "not good" or "very happy". But they come with limitations:
  - CNNs only capture fixed-size n-gram features.
  - They struggle with long-distance dependencies in language.
    - Example: “The book I bought yesterday was amazing.” → “book” and “amazing” are far apart.
  - CNNs are position-invariant, meaning they don't inherently understand word order.
- To model sequence and context over time, we need a model that remembers previous inputs—this is where **RNNs** come in.

Recurrent Neural Networks Introduction



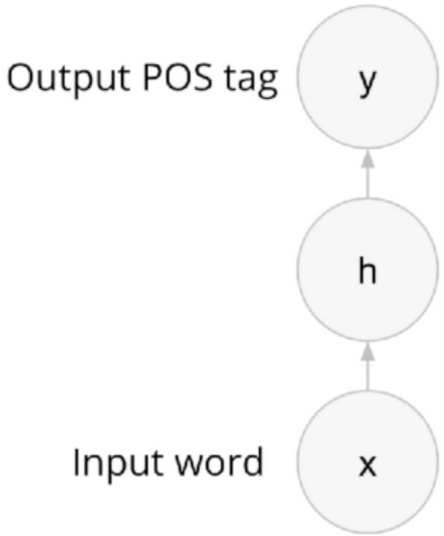
Recurrent Neural Networks Introduction



傳遞給下一個時間
點的自己

Why should we use RNNs?

- Consider classifying words (e.g. parts-of-speech: noun, verb, etc.)
- Suppose we use an ANN:

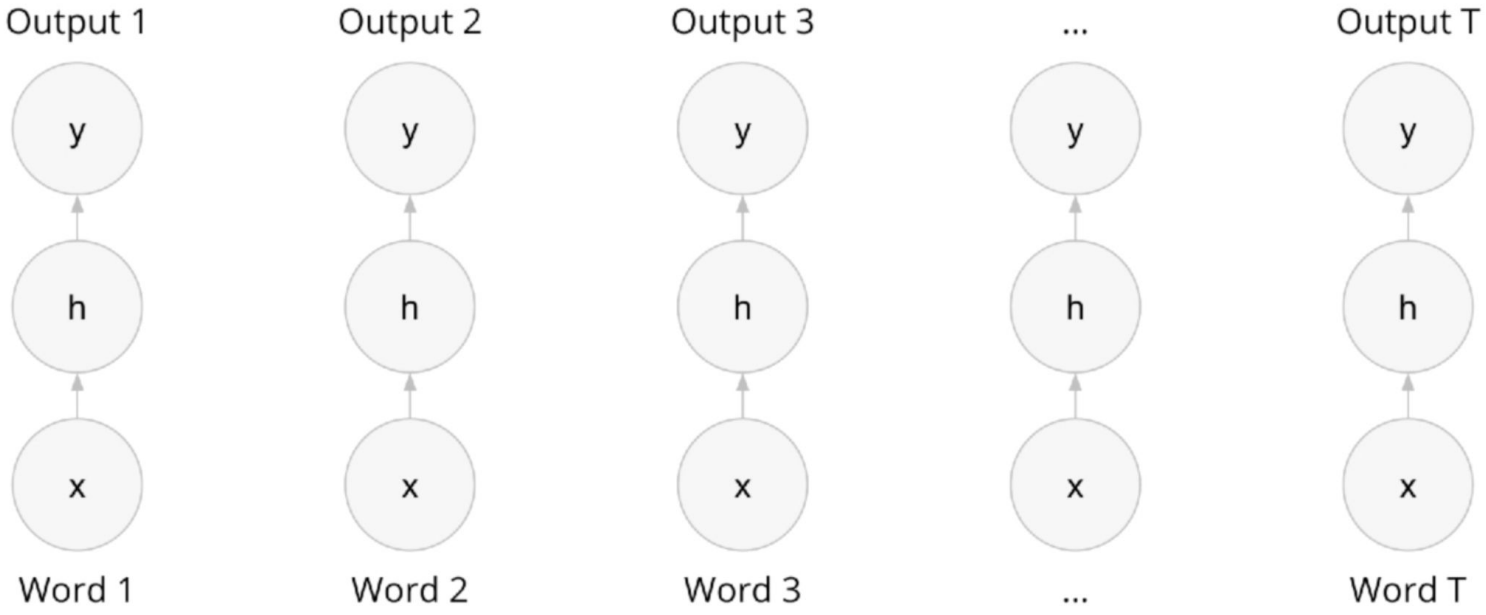


Problem with ANN

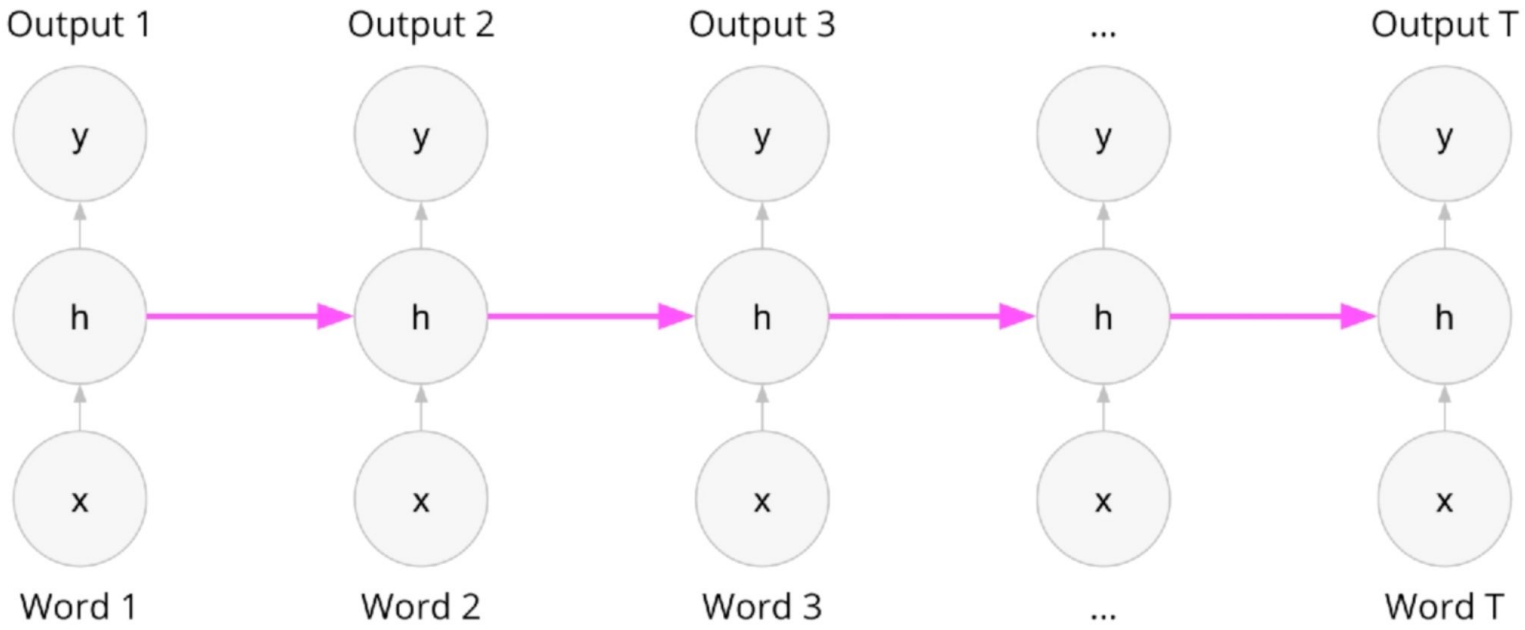
- Sometimes there can be multiple possible answers, which can only be disambiguated with context
- E.g. “bank” has several meanings
  - Financial institution (maybe higher chance of spam)
  - “River bank” (maybe lower chance of spam)
  - As a verb (and still has multiple meanings)

“Your bank has requested your password.”
“Let’s take a walk on the river bank.”
“Would you bank on it?”
“I bank with Bank of America.”

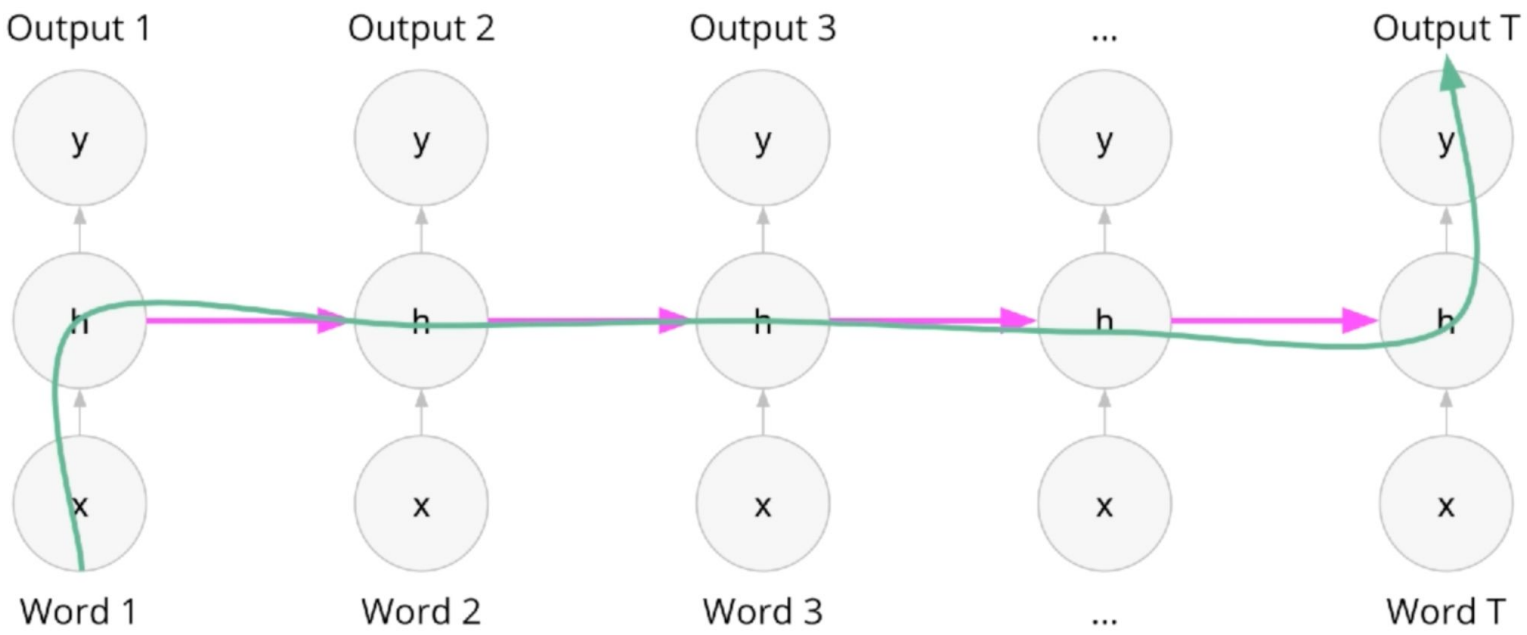
Back to Our Classification Problem - ANN



Back to Our Classification Problem -RNN



Back to Our Classification Problem -RNN



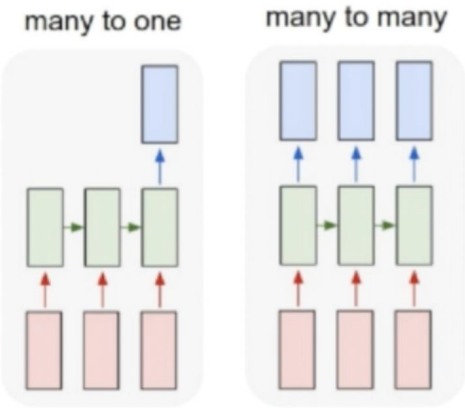
Problem Types RNNs Can Solve

Many-to-One

- Definition: Multiple inputs → Single output
- Examples:
  - Spam detection
  - Sentiment analysis

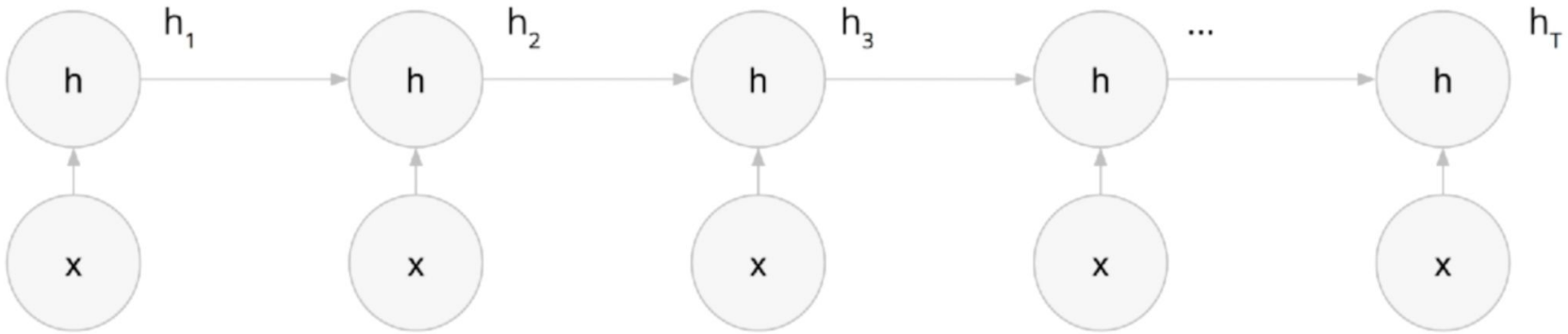
Many-to-Many

- Definition: Each input → corresponding output
- Examples:
  - Part-of-speech tagging
  - Time series anomaly detection



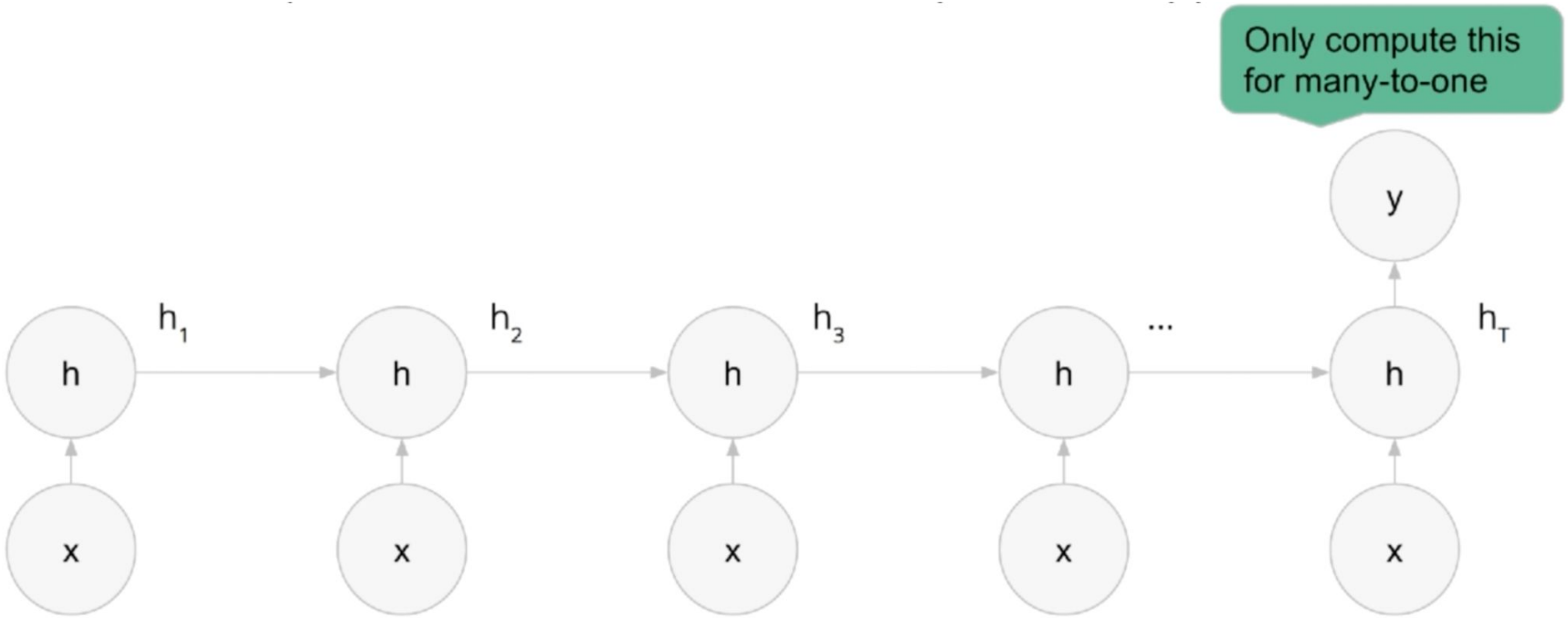
What Do We Do with h(t)?

- h(t) is the output of RNN at each time step



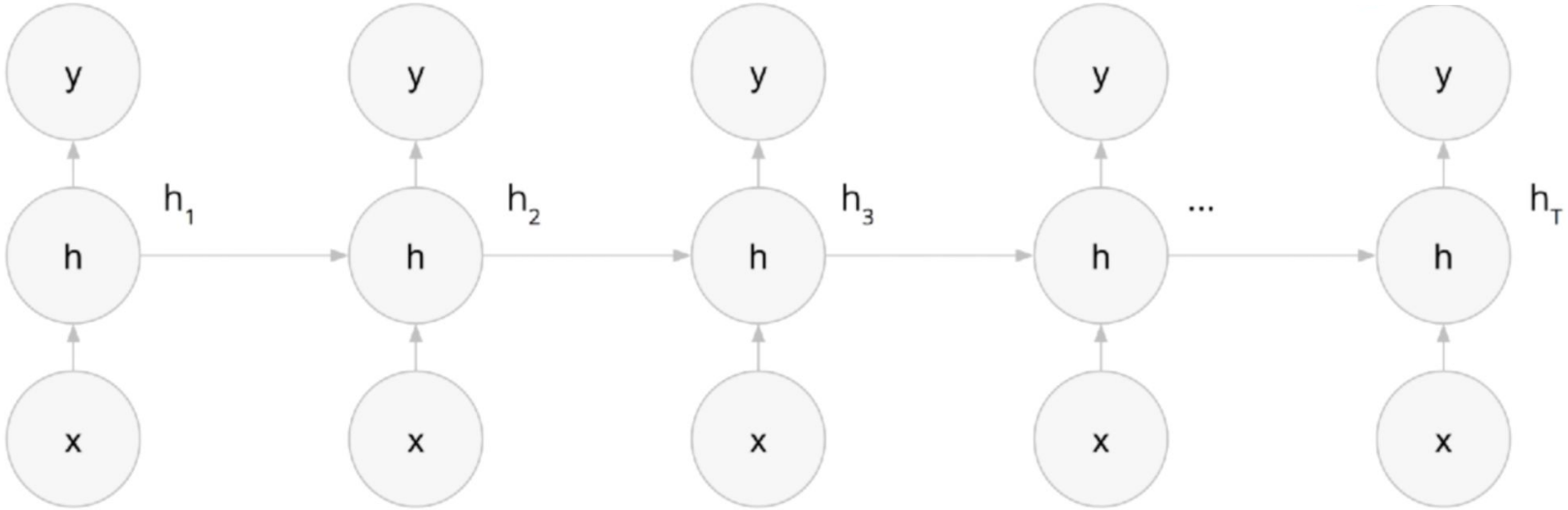
What Do We Do with h(t)?

- Many-to-One



What Do We Do with h(t)?

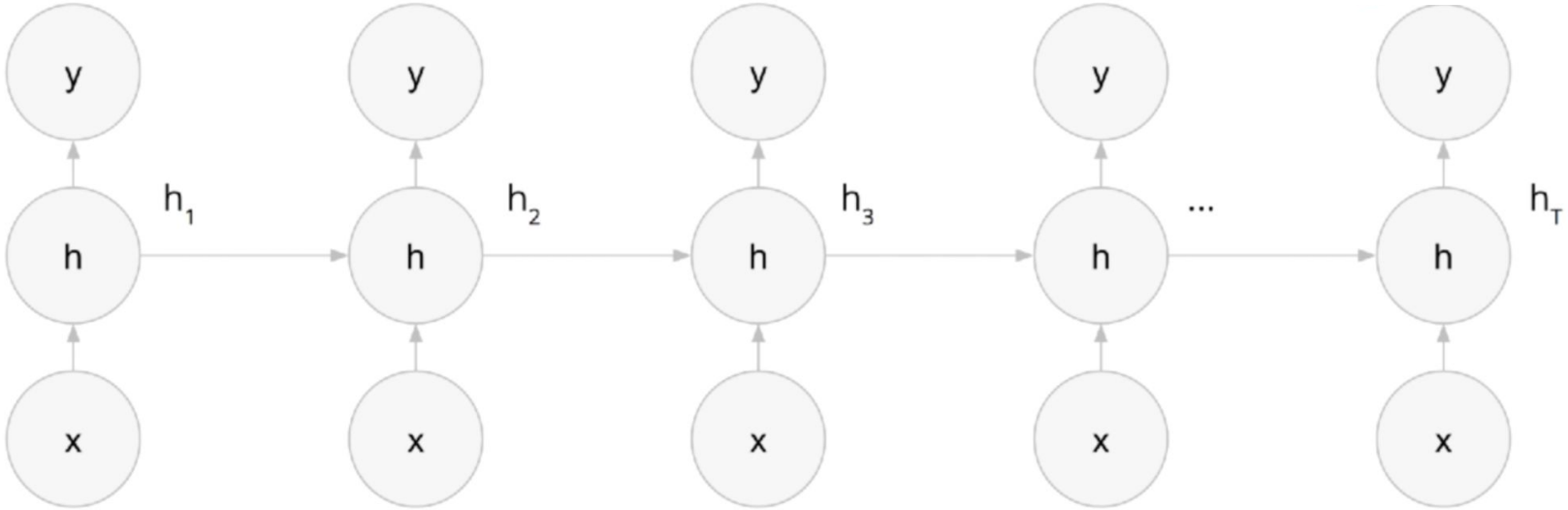
- Many-to-Many



What Do We Do with h(t)?

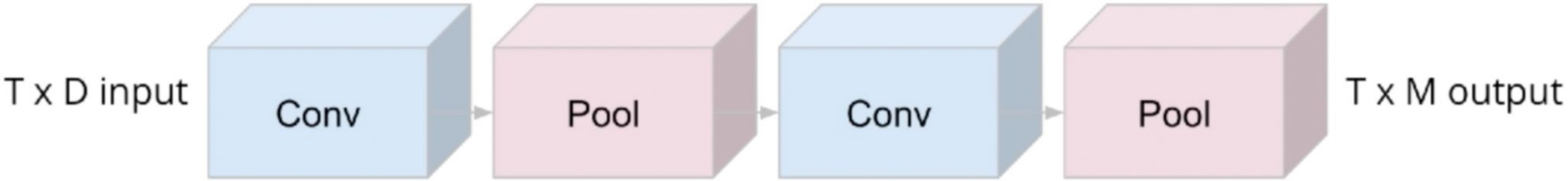
- Many-to-Many

Note: the same weights are used at every time step for each layer (weight sharing)



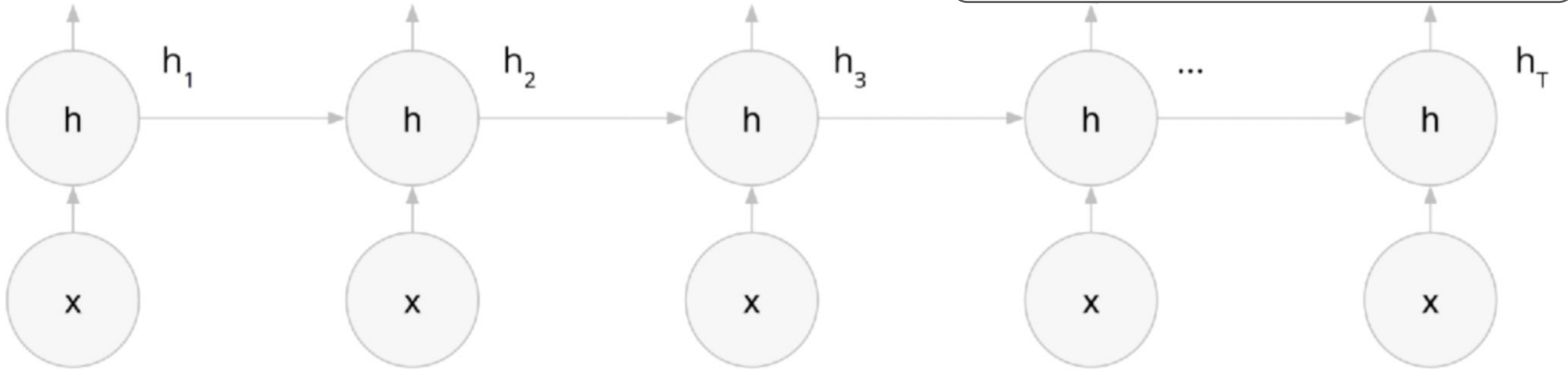
Other Options for Many-to-One

- CNN

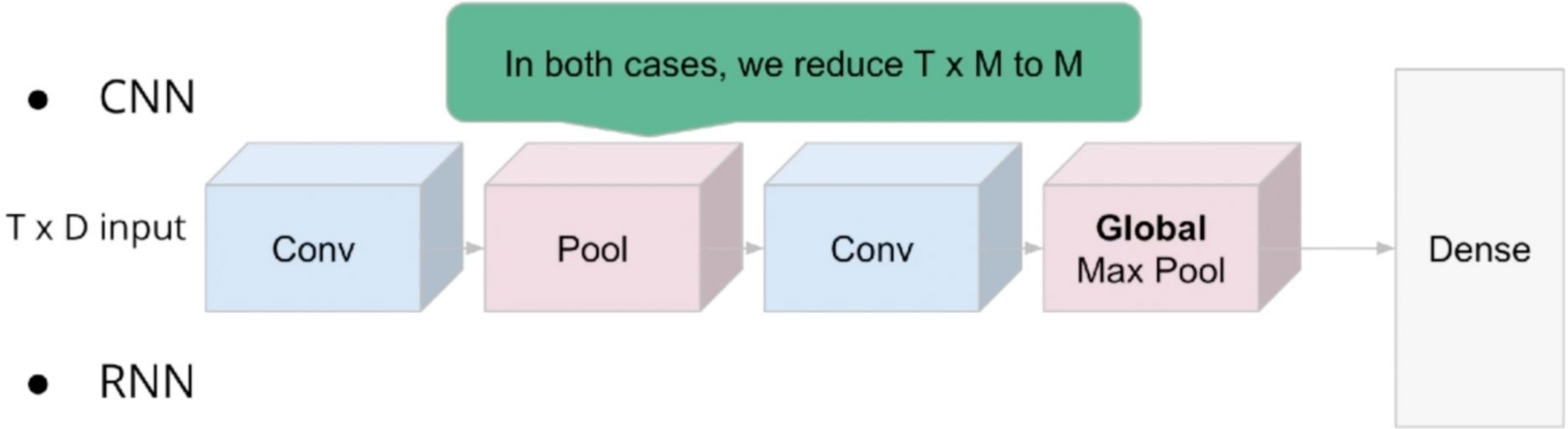


- RNN

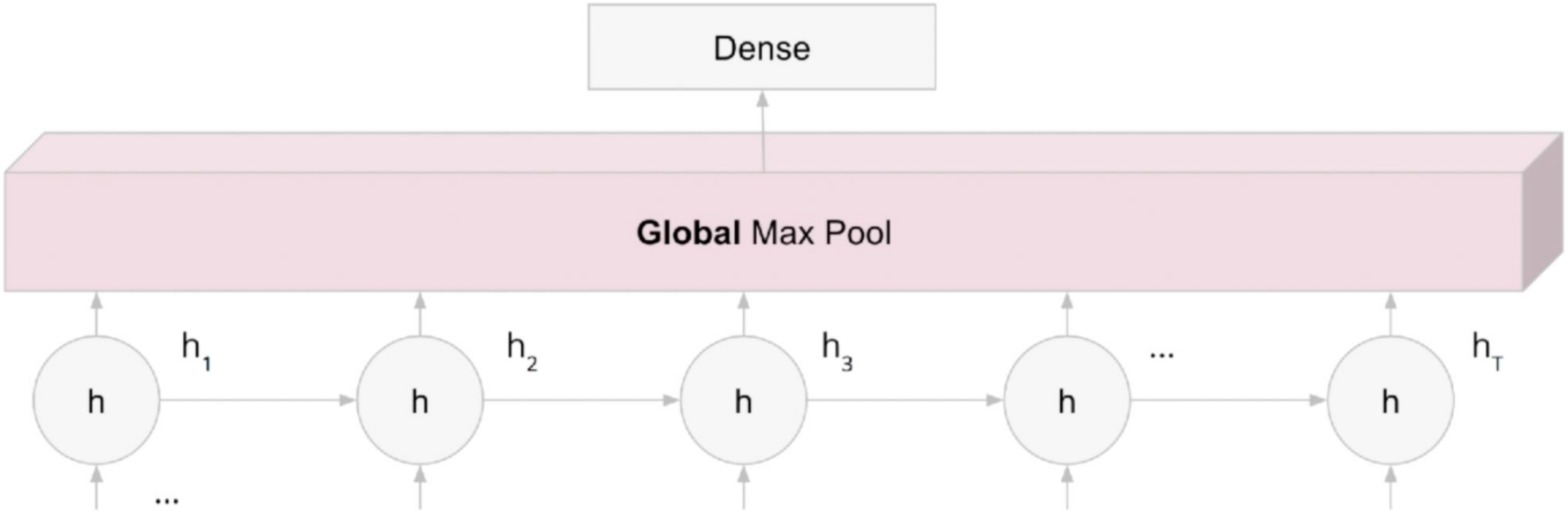
Also T x M output, since we have T hidden vectors each of size M



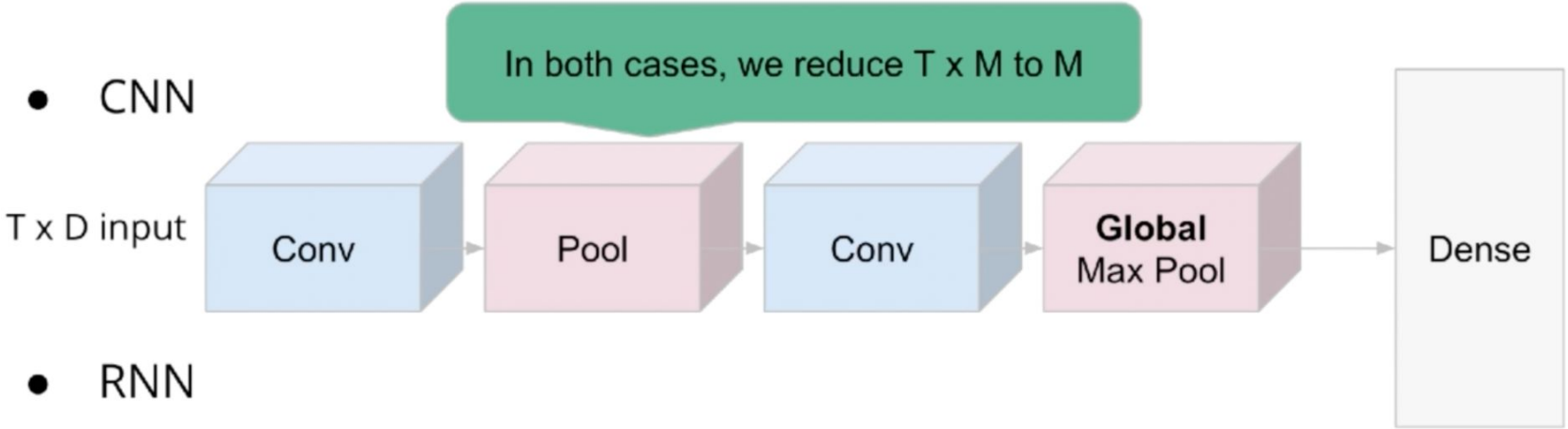
- CNN


In both cases, we reduce T x M to M
T x D input
Conv
Pool
Conv
Global
Max Pool
Dense

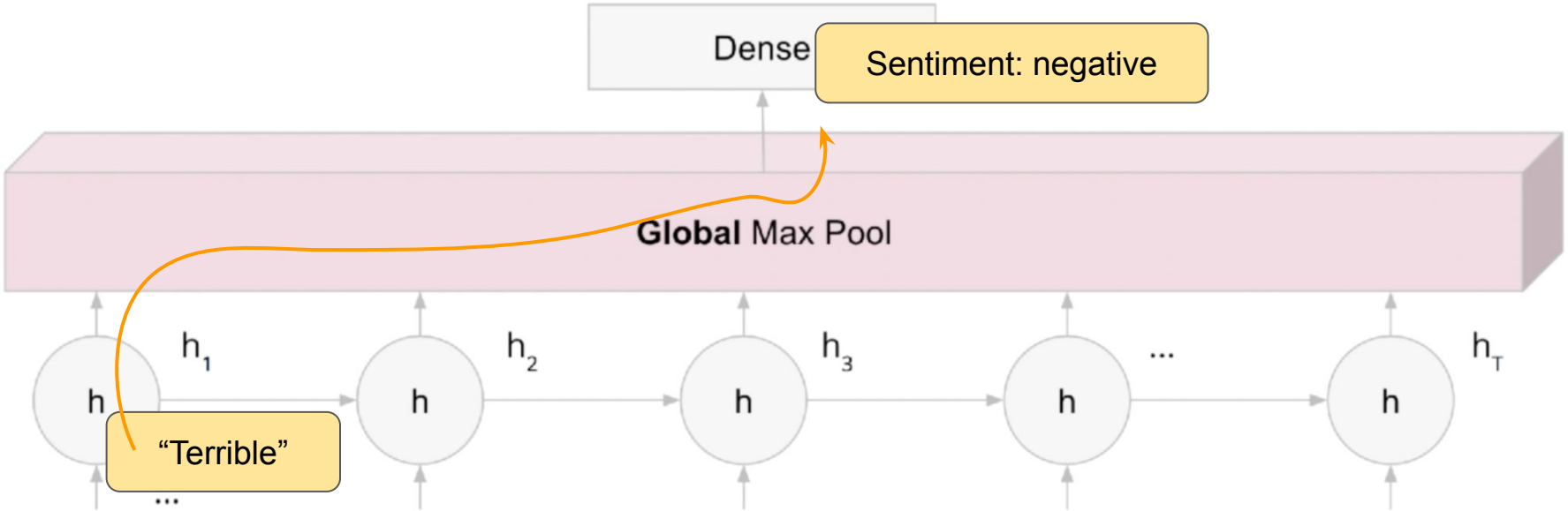
- RNN


Dense
Global Max Pool
h
h
h
h
h
h1
h2
h3
...
hT
...

- CNN

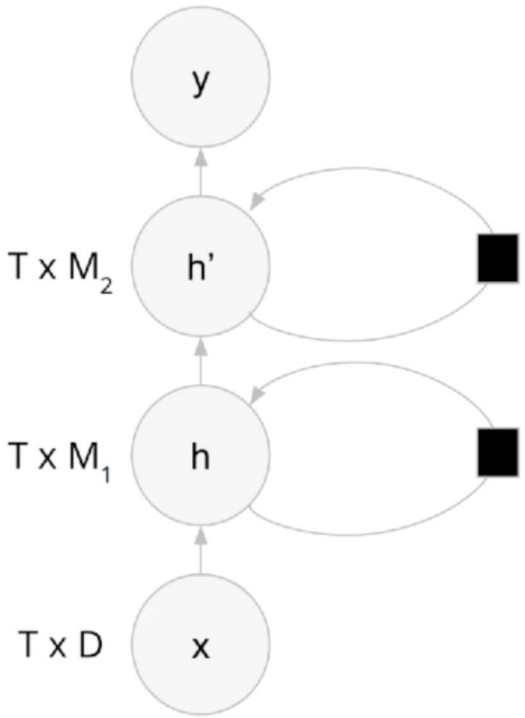

In both cases, we reduce T x M to M
T x D input
Conv
Pool
Conv
Global
Max Pool
Dense

- RNN


Dense
Sentiment: negative
Global Max Pool
h
h
h
h
h
h1
h2
h3
...
hT
"Terrible"
...

Stacking RNN Layers

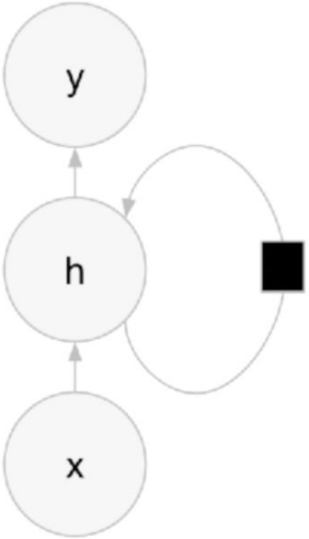
- Note: one common mistake is confusing T with M
- The **sequence length (T)** is not the same as the **size of the hidden vectors (M)**
  (each circle does not represent a scalar)
- Normally we would not vary the size of the hidden layers (e.g. normally M1 = M2)



RNN in Tensorflow

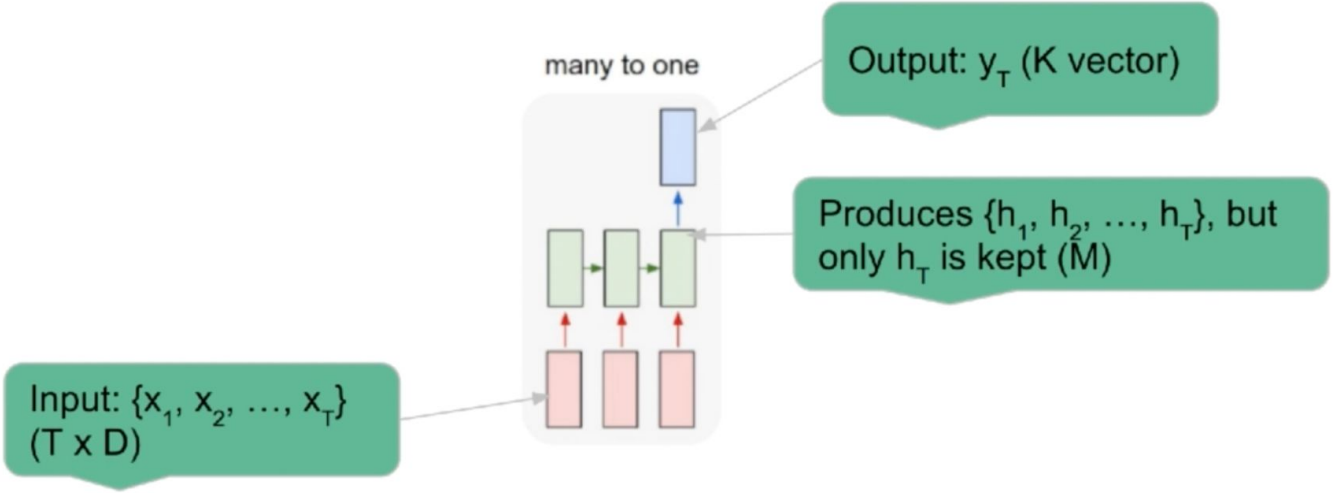
```
i = Input(shape=(D,))
x = Dense(M, activation='relu')(i)
x = Dense(K)(x)
model = Model(i, x)
```

```
i = Input(shape=(T, D))
x = SimpleRNN(M)(i)
x = Dense(K)(x)
model = Model(i, x)
```



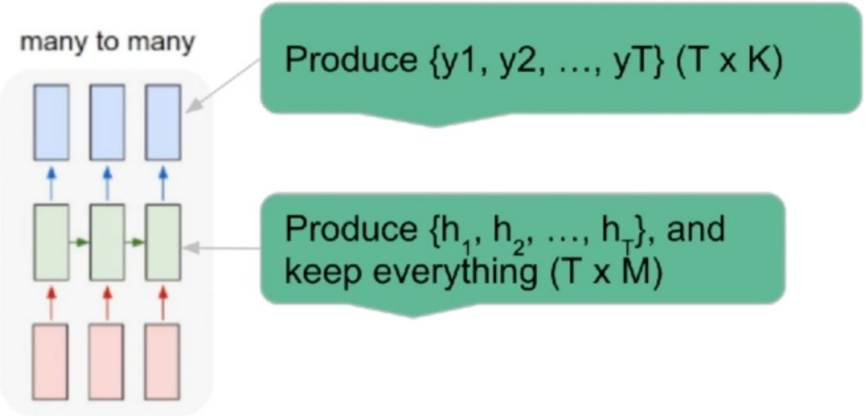
Output: Many-to-One

```
i = Input(shape=(T, D))
x = SimpleRNN(M)(i)
x = Dense(K)(x)
model = Model(i, x)
```



Output: Many-to-Many

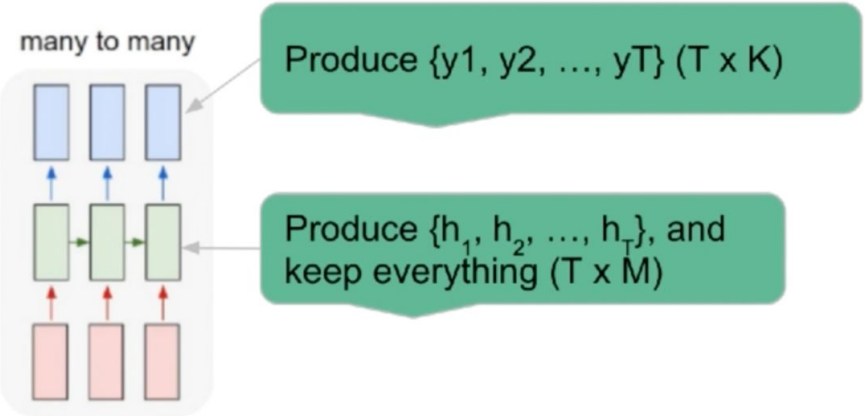
```
i = Input(shape=(T, D))
x = SimpleRNN(M, return_sequences=True)(i)
x = Dense(K)(x)
model = Model(i, x)
```



Output: Many-to-Many

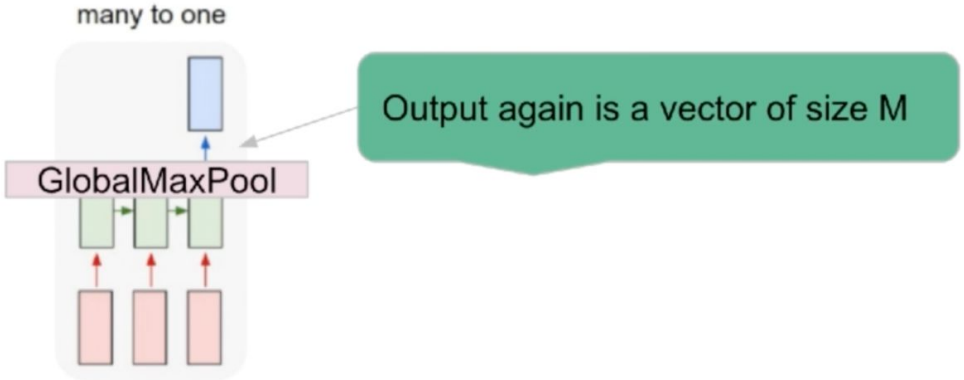
```
i = Input(shape=(T, D))
x = SimpleRNN(M, return_sequences=True)(i)
x = Dense(K)(x)
model = Model(i, x)
```

The Dense layer “knows” how to handle a single vector and a time series of vectors as input



Output: Many-to-One (another way)

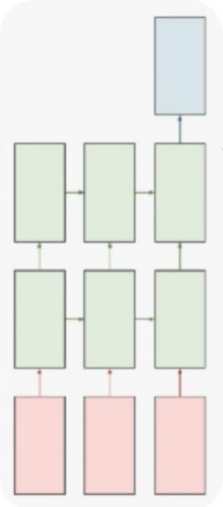
```
i = Input(shape=(T, D))
x = SimpleRNN(M, return_sequences=True)(i)
x = GlobalMaxPooling1D()(x)
x = Dense(K)(x)
model = Model(i, x)
```



Stacking Layers

```
i = Input(shape=(T, D))
x = SimpleRNN(32, return_sequences=True)(i)
x = SimpleRNN(32)(x) # default is return_sequences=False
x = Dense(K)(x)
model = Model(i, x)
```

many to many stacked



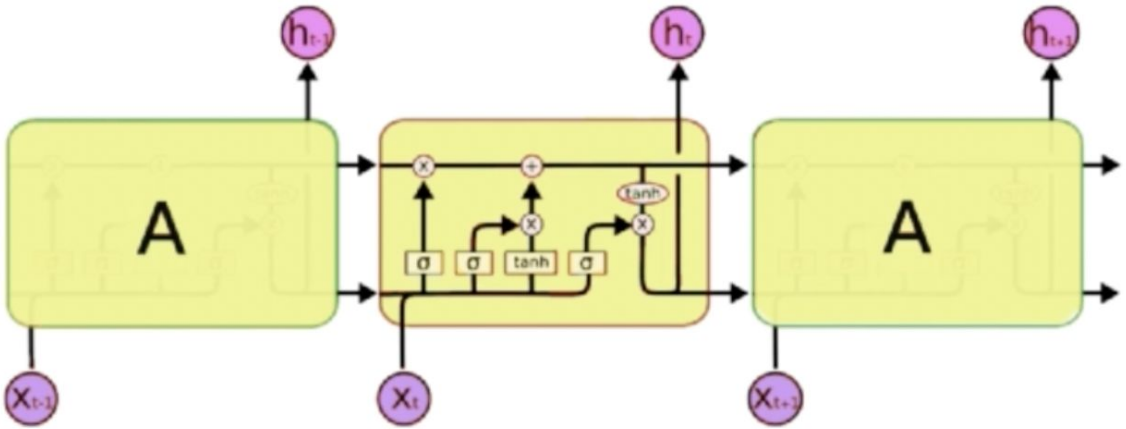
Easy to Use LSTMs and GRUs (Preview)

```
i = Input(shape=(T, D))
x = LSTM(M)(i)
x = Dense(K)(x)
model = Model(i, x)
```

```
i = Input(shape=(T, D))
x = GRU(M)(i)
x = Dense(K)(x)
model = Model(i, x)
```

Modern RNN Units

- LSTM - Long Short-Term Memory
- GRU - Gated Recurrent Unit
- GRU is like a simplified version of the LSTM (less parameters and thus more efficient)



LSTM

Limitations of Vanilla RNNs

- While RNNs can capture word order and dependencies, they have their own issues:
  - **Vanishing gradients**: Difficult to learn long-range dependencies (early inputs get "forgotten")
  - **Slow to train**: Because each step depends on the previous one, RNNs are not parallelizable

Vanishing gradients: https://www.youtube.com/watch?v=8z3DFk4VxRo

How Does the Problem Manifest?

- Ask your simple RNN: “On what day was Albert Einstein born?”
- By the time the RNN has “read” the whole article, it forgot what was at the beginning!

Albert Einstein

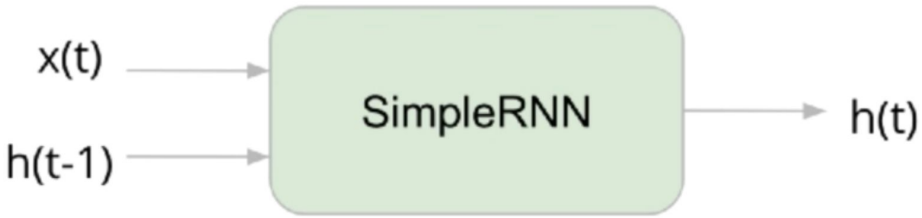
From Wikipedia, the free encyclopedia

"Einstein" redirects here. For other people, see Einstein (surname). For other uses, see Albert Einstein (disambiguation) and Einstein (disam

Albert Einstein (/ˈaɪnstaɪn/ *EYEN-styne*;[4] German: [ˈalbɛɐ̯t ˈʔaɪnʃtaɪn] (listen); 14 March 1879 – 18 April 1955) was a German-born theoretical physicist[5] who developed the theory of relativity, one of the two pillars of modern physics (alongside quantum mechanics).[3][6]:274 His work is also known for its influence on the philosophy of science.[7][8] He is best known to the general public for his mass–energy equivalence formula $E = mc^2$, which has been dubbed "the world's most famous equation".[9] He received the 1921 Nobel Prize in Physics "for his services to theoretical physics, and especially for his discovery of the law of the photoelectric effect",[10] a pivotal step in the development of quantum theory.

Gated Recurrent Unit (GRU)

- First and foremost, it has the same “API” as the simpleRNN



GRU

Simple RNN unit, for comparison

$$h_t = \tanh(W_{xh}{}^T x_t + W_{hh}{}^T h_{t-1} + b_h)$$

$$z_t = \sigma(W_{xz}{}^T x_t + W_{hz}{}^T h_{t-1} + b_z)$$

$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

GRU

Simple RNN unit, for comparison

$$h_t = \tanh(W_{xh}{}^T x_t + W_{hh}{}^T h_{t-1} + b_h)$$

$$z_t = \sigma(W_{xz}{}^T x_t + W_{hz}{}^T h_{t-1} + b_z)$$

$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

z(t) = update gate vector
r(t) = reset gate vector
h(t) = hidden state

GRU

- z(t), r(t), h(t) are all vectors of size M
- M is a hyperparameter (“number of hidden units / features”)
- This implies the shape of all the weights
- Any weight going from x(t) is D x M
- Any weight going from h(t) is M x M
- All bias terms are of size M

$$z_t = \sigma(W_{xz}{}^T x_t + W_{hz}{}^T h_{t-1} + b_z)$$
$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

What is the GRU Doing?

- z(t) = update gate vector
- “Should I take the new value for h(t), or should I keep the old value, h(t-1)?”
- Vanishing gradient means the RNN forgets things in the past
- Now, we can explicitly remember the previous h(t-1) (If z(t) small)
- If z(t) → 1, then forget h(t-1)

Circle with a dot is element-wise multiplication

$$z_t = \sigma(W_{xz}{}^T x_t + W_{hz}{}^T h_{t-1} + b_z)$$

$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$

$$h_t = \boxed{(1 - z_t) \odot} h_{t-1} + \boxed{z_t \odot} \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

Interpreting the GRU

- (Forget about the reset gate for now)

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

$$h_t = p(keep\ h_{t-1}) h_{t-1} + p(discard\ h_{t-1}) SimpleRNN(x_t,\ h_{t-1})$$

Reset Gate

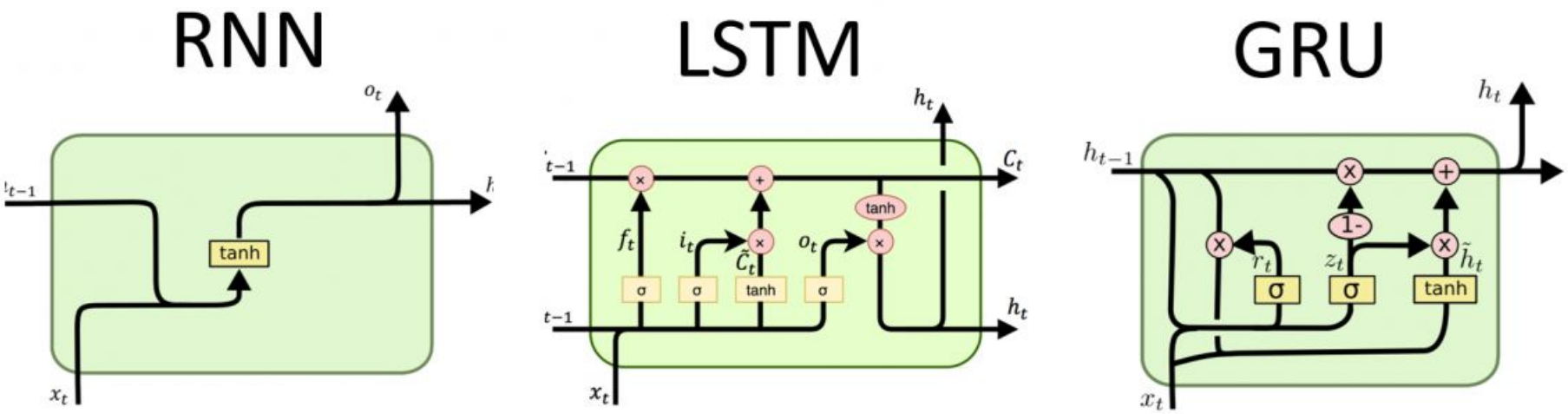
- Reset gate → whether to look up old data when generating new hidden state
- Update gate → whether to update the database now

$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T \boxed{(r_t} \odot h_{t-1}) + b_h)$$

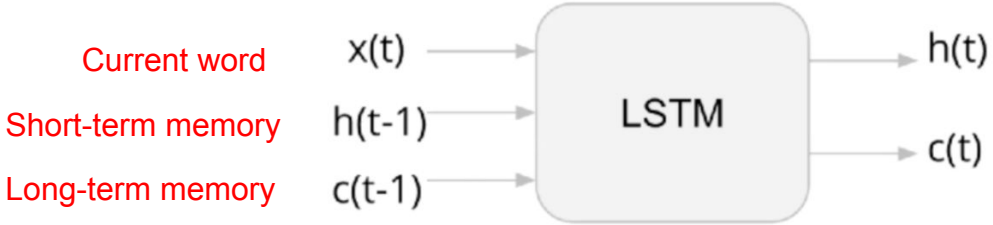
LSTM (Long Short-Term Memory)

- Think of it as “like the GRU” but with more state vectors and gates



GRU vs LSTM

- Not exactly the same API
- LSTM returns 2 states:
  - Hidden state h(t)
  - Cell state c(t) (usually ignored)
  - Also means you need 2 initial states: c_0 and h_0



LSTM Equations

$$f_t = \sigma(W_{xf}{}^T x_t + W_{hf}{}^T h_{t-1} + b_f)$$

Forget gate

$$i_t = \sigma(W_{xi}{}^T x_t + W_{hi}{}^T h_{t-1} + b_i)$$

Input/update gate

$$o_t = \sigma(W_{xo}{}^T x_t + W_{ho}{}^T h_{t-1} + b_o)$$

Output gate

$$c_t = f_t \odot c_{t-1} + i_t \odot \underbrace{f_c}_{\text{tanh}}(W_{xc}{}^T x_t + W_{hc}{}^T h_{t-1} + b_c)$$

$$h_t = o_t \odot \underbrace{f_h}_{\text{tanh}}(c_t)$$

"Simple RNN"

Why Output Gate?

- 為何不直接將c_t的值輸出作為h_t?
- 長期記憶通常包含豐富且雜亂的資訊，直接輸出的話下一層會被迫接收大量當下根本不需要的雜訊
- Ex: “Alice 是一個非常優秀的工程師，我昨天在會議上遇到她”（translation task）
  - 當模型讀到Alice時，c_t 會記住兩個特徵：女性、單數
  - 當模型讀到“是一個非常優秀的工程師 ...”時，後面的預測層根本不需要知道 Alice 是女性這件事
  - 這時，output gate會關閉對於性別特徵的輸出，不讓它干擾當下的語意判斷
  - 直到最後讀到”遇到...”，準備要預測代名詞時，output gate才會打開，把c_t裡的”女性”特徵釋出到h_t，讓模型能夠預測出“her。

LSTM is Simple

f(t) = Neuron (binary classifier)

i(t) = Neuron (binary classifier)

o(t) = Neuron (binary classifier)

c(t) = f(t) * c(t-1) + i(t) * SimpleRNN

h(t) = o(t) * tanh(c(t))